ЛР №11 — дослідженська частина (§4). ShowFinder і TVmaze API

Тема: AI-агент для інтеграції REST API: Fetch API, Promise та async/await, аналіз API-документації, loading та error states, debugging через Network.

Репозиторій	https://github.com/HunterDead14/Web_React (тека `Laba_11/`)
Опублікована адреса	https://hunterdead14.github.io/Web_React/Laba_11/ · самоперевірка: …`/Laba_11/check.html`
Конспект API	*`Laba_11/API.md`* *(./API.md)* — §4.5
Правила для агента	`Laba_11/AGENTS.md` — буде створено після аудиту C1–C2 (ТЗ, §4)
Дата роботи	19.10.2026
AI-агент	OpenCode (Kodeo/OpenCode CLI), режим агента з редагуванням файлів
Модель	`minimax-m3.1-flash-free` (провайдер `opencode`)
Термін здачі	22.10.2026

> ***Як отримано результати.*** *Усі 13 експериментів прогнано в* ***справжньому Chrome*** *на сторінці* `Laba_11/experiments.html`*, піднятій локальним сервером (*`http://127.0.0.1:8123/experiments.html`*).*
> *Console і панель Network знімались програмно з того самого браузера, тому в колонці «Фактичний результат» — дослівний вивід, а не переказ. Кожен експеримент виконувався на* ***чистій сторінці*** *(новий таб), щоб стан кешу й Console не перетікав між тестами.*
>
> ⚠️ ***Про колонку «Мій аналіз».*** *Це чернетка аналізу, зібрана з нотаток під час підготовки. §5.2 лабораторної вимагає: «Мій аналіз пишете самі, без AI-агента й до запуску». Перед здачею перепиши цю колонку* ***своїми словами*** *— щоб це був твій аналіз, а не конспект. Фактичні результати й розбіжності вже перевірені — їх не треба змінювати.*

1. Таблиця експериментів (§6, п. 1 звіту)

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)

1	**fetch і статус 404.** `GET /shows/999999999`	`fetch` не знає про HTTP-статуси як про помилки: він знає лише «мережа працює / не працює». Статус — просто число в `Response`. Тому проміс не відхилиться, а `catch` не спрацює. Очікував `ok=false, status=404` і JSON-тіло помилки TVmaze.	**Console:**
			`false 404`
			`{name: 'Not Found', message: '', code: 0, status: 404}`
			червоний рядок: `Failed to load resource: the server responded with a status of 404 ()`
			Network: 1 рядок, `GET https://api.tvmaze.com/shows/999999999` → **404**, 276 мс
				Розбіжність: я прогнозував «у Console нічого червоного» — **є червоний рядок**. Але він від браузера (мережевий лог), а не від нашого коду: наш код відпрацював штатно. Черговий рядок не означає, що «код упав» — саме це й питає лабораторна.

2	**Тіло відповіді читається один раз** + HTML замість JSON	Тіло — одноразовий потік байтів; після першого читання він «спустошений». Друге читання — помилка стану об'єкта, а не HTTP-помилка. HTML з JSON-парсером дає іншу помилку: `SyntaxError` на першому символі.	**Console:**
			`Game of Thrones`
			`вдруге: TypeError | Failed to execute 'json' on 'Response': body stream already read`
			`HTML content-type: text/html | status 200`
			`HTML: SyntaxError Unexpected token '<', "<!doctype "... is not valid JSON`
			Network: `GET /shows/82` → 200; `GET experiments.html` → 200, `content-type: text/html`
				Розбіжності:
				1) `id=82` — це **Game of Thrones**, а не «The Cosby Show», як я записав у чернетці.
				2) Текст помилки у вашому Chrome: `body stream already read` (Chrome 152), а не «Body is unusable» — це лише формулювання.
				3) Різниця `TypeError` (прочитав двічі) проти `SyntaxError` (сервер дав не JSON) — підтвердилася.

3	**`forEach`, `for…of` і `Promise.all`**	`forEach` не чекає на проміси: він синхронно проходить масив, тому `counts.length` = 0 одразу. `for…of` з `await` — послідовно, `Promise.all` — паралельно, тому друга дорівняє сумі затримок, а третя — максимальній.	**Console:**
			`forEach: 0`
			`через 2 с: 3`
			`for…of: 208 мс`
			`Promise.all: 155 мс`
			Network (Waterfall), 9 запитів:
			• `forEach`: старт `0/1/1 мс` → **паралельно**
			• `for…of`: старт `2003 → 2093 → 2143 мс` → **поспіль**
			• `Promise.all`: старт `2212/2212/2212 мс` → **паралельно**
				Розбіжність: я прогнозував ≈**3×** різницю, фактично **208 проти 155 мс ≈ 1,3×**. Причина — мала затримка й warm-з'єднання: кожен запит у послідовному варіанті уже «на гарячому» каналі. **Структурно** Waterfall доводить головне: поспіль проти паралельно.

4	**`return` і `return await` у `try`**	`try/catch` ловить синхронні винятки. `return fetch(url)` — це повернення проміса: `try` завершується **до** відхилення, тому помилка «проходить мимо» локального `catch` і дістається до зовнішнього `.catch`.	**Console:** `поза load: TypeError` (варіант `return fetch(url)`) `catch у load: TypeError` (варіант `return await fetch(url)`) `варіант із return await -> null` **Network:** 2 запити до `https://api.tvmaze.invalid/shows/82` → `failed`, `net::ERR_TUNNEL_CONNECTION_FAILED`	Підтвердилося. **Уточнення для звіту:** у нашому середовищі браузер іде через проксі-тунель, тому помилка — `ERR_TUNNEL_CONNECTION_FAILED`; у звичайному браузері без проксі буде `net::ERR_NAME_NOT_RESOLVED` (домен `.invalid` не існує). Для JS-коду це одна й та сама помилка: `TypeError` / `Failed to fetch`.

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)

5	**`Promise.all` і `Promise.allSettled`**	`Promise.all` відхиляється на першій помилці й **відкидає** решту — дві вдалі відповіді, які вже прийшли, втрачено. `allSettled` не відхиляється ніколи: кожен результат має `{ status, value | reason }`.	**Console:**
			`all: TypeError | Failed to fetch`
			`allSettled: ["fulfilled","rejected","fulfilled"]`
			`значення fulfilled: 200,200`
			`причина rejected: TypeError`
			Network: у фазі `all` бачимо `GET /shows/82` → **200** і `GET /shows/139` → **200** (обидва вдали!), але результату немає — проміс уже відхилився.
				Підтвердилося без застережень. `allSettled` дав рівно `["fulfilled","rejected","fulfilled"]`.

6	**`AbortController`**	Скасування до відповіді → `AbortError`. Скасування **після** отримання заголовків не обнуляє вже отриманий `Response` — `status` і `ok` лишаються дійсними, але тіло вже не прочитається.	**Console:**
			`до відповіді: AbortError | signal is aborted without reason`
			`після відповіді: 200 | ok: true | type: cors`
			`json(): AbortError | The user aborted a request.`
			Network: `GET /search/shows?q=girls` → `net::ERR_ABORTED`; `GET /shows/139` → **200**, але `failed / net::ERR_ABORTED`
				Підтвердилося. **Уточнення:** другий запит устиг отримати статус 200 (заголовки прийшли), але читання тіла обірвалося — саме це і показує `json(): AbortError`. У панелі Network такий запит має статус **`(canceled)`**.

7	**Заголовки відповіді**	Браузер показує скрипту лише CORS-safelisted заголовки: `Cache-Control`, `Content-Language`, `Content-Length`, `Content-Type`, `Expires`, `Last-Modified`, `Pragma`. `Date`, `Server`, `Retry-After` приховані, поки сервер не додасть `Access-Control-Expose-Headers`.	**Console:**
			`content-type: application/json; charset=UTF-8`
			`cache-control: public, max-age=3600`
			`date: null`
			`server: null`
			`retry-after: null`
			`ключі: Array(2) → ['cache-control', 'content-type']`
			Network → Response Headers (DevTools бачить усе):
			`access-control-allow-origin: *`
			`cache-control: public, max-age=3600`
			`content-encoding: gzip`
			`content-type: application/json; charset=UTF-8`
			`date: Mon, 19 Oct 2026 20:54:38 GMT`
			`server: nginx/1.24.0 (Ubuntu)`
			`vary: Accept-Encoding`
				Підтвердилося. **Уточнення:** скрипт бачить **рівно 2** заголовки, а не 3 — `content-length` відсутній, бо TVmaze віддає `transfer-encoding: chunked` (і `content-encoding: gzip`). `date` і `server` сервер надсилає, але до скрипта не доходять.

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)

8	**HTTP-кеш**	`Cache-Control: public, max-age=3600` → браузер зберігає відповідь на годину; другий запит не йде в мережу. `cache: 'no-store'` = не читати кеш **і** не зберігати.	**Console (час виконання):**
			`1 звичайний: 159 мс, статус 200`
			`2 звичайний: 6 мс, статус 200`
			`3 no-store: 57 мс, статус 200`
			`4 звичайний (повтор): 6 мс, статус 200`
			Network: та сама картина — 1-й запит 156 мс, 2-й 4 мс, 3-й 54 мс, 4-й 4 мс
				Підтвердилося. **Уточнення (цікаве):** 4-й запит знову з кешу — `no-store` **не перезаписує** вже збережену відповідь, а лише не читає і не зберігає її. Тому після `no-store` звичайний запит знову миттєвий.

9	**Переадресація**	`fetch` сам іде за переадресацією, тому клієнт ніколи не бачить 301. Після переадресації `status=200`, `redirected=true`, а `url` — фінальна адреса.	**Console:**
			`200 true https://api.tvmaze.com/shows/82`
			`Game of Thrones`
			Network — два рядки:
			`GET https://api.tvmaze.com/lookup/shows?imdb=tt0944947` → **301** (64 мс)
			`GET https://api.tvmaze.com/shows/82` → **200** (3 мс)
				Підтвердилося. **Уточнення:** IMDb-id `tt0944947` — це **Game of Thrones**, TVmaze-id **82** (той самий, що в експерименті 2).

10	**Рядок запиту**	`&` у рядку запиту — роздільник параметрів, тож `Law & Order` без кодування розривається на `q=Law` і окремий параметр `Order`. `URLSearchParams` кодує значення сам.	**Console:**
			`Law  Array(2)` → `q = "Law "`, ключів **2**
			`Law & Order ?q=Law+%26+Order`
			`сирий рядок plain: https://api.tvmaze.com/search/shows?q=Law%20&%20Order`
			`сирий рядок encoded: https://api.tvmaze.com/search/shows?q=Law+%26+Order`
				Підтвердилося без застережень. Перша адреса має **два** параметри (`q` і порожній `Order`), друга — один, але правильно закодований.

11	**Два домени TVmaze**	`api.tvmaze.com` і `www.tvmaze.com` — різні origin (протокол + домен + порт), хоча домен один. Запит на інший origin без `Access-Control-Allow-Origin` блокується **браузером** — і блокується саме *читання* відповіді, а не надсилання.	**Console:**
			`api: 200`
			червоний рядок: `Access to fetch at 'https://www.tvmaze.com/shows/82' from origin 'http://127.0.0.1:8123' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.`
			`www: TypeError Failed to fetch`
			Network: рядок `GET https://www.tvmaze.com/shows/82` є, `failed / net::ERR_FAILED` (236 мс)
				Уточнення до формулювання: запит **дійшов** до сервера, і відповідь **прийшла** — інакше браузер не зміг би написати «No 'Access-Control-Allow-Origin' header is present»: цю помилку він може дати лише після отримання відповіді. Заблоковано доступ скрипта до її тіла. У панелі Network рядок позначено `failed / net::ERR_FAILED`.

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)

12	**Preflight**	«Простий» запит — це GET/HEAD/POST + лише safelisted-заголовки. `Content-Type: application/json` — **не** safelisted, тому браузер спершу надсилає `OPTIONS` (preflight) і чекає дозволу. `Accept` — safelisted, preflight не потрібен.	**Console:**
			червоний рядок: `Access to fetch at 'https://api.tvmaze.com/shows/82' ... Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present ...`
			`Content-Type: TypeError Failed to fetch`
			`Accept: 200`
			Network — три рядки:
			`GET /shows/82` → `failed / net::ERR_FAILED`
			`OPTIONS /shows/82` → **405** (98 мс)
			`GET /shows/82` → **200** (3 мс, це вже запит з `Accept`)
				Підтвердилося. **Уточнення:** на перший запит припадає **два** рядки — `OPTIONS 405` і провалений `GET`; сам `GET` з `Content-Type` **не дійшов** до сервера. Другий запит (з `Accept`) — один рядок, без `OPTIONS`. Тобто «All endpoints are CORS enabled» означає дозвіл **простих** запитів, а не будь-яких заголовків.

13	**`mode: 'no-cors'`**	`no-cors` не «виправляє» CORS, а посилює обмеження: відповідь стає **opaque** — браузер приховує від скрипта все (статус, заголовки, тіло). Помилка зникає разом із даними.	**Console:**
			`opaque 0 false`
			`""`
			Network: `GET https://www.tvmaze.com/shows/82` → **301**, далі `GET https://www.tvmaze.com/shows/82/game-of-thrones` → `failed / net::ERR_ABORTED`
				Підтвердилося: `type='opaque'`, `status=0`, `ok=false`, тіло — **порожній рядок `""`** (не помилка!). **Цікавий дрібниць:** сайт TVmaze віддає `301` на канонічну адресу `/shows/82/game-of-thrones` — і цю переадресацію браузер простежує і в `no-cors`-режимі, але скрипту нічого про неї не каже.